

Python Coding Standard — v2.0

Owner: Kishore Rayan · **Approved:** Praveen (CTO)

Stack: FastAPI · SQLAlchemy · Alembic · Pydantic v2

Supersedes: Python Coding Standard v1.0

Scope and status

This standard applies to **new and changed code**. Existing code is not retrofitted — a rule arriving here does not create a backlog of rewrites. When you touch a file, the code you write in it follows this document.

Where a rule has sample code, that code is the house pattern. Copy it rather than writing your own version.

Sections 1–12 mirror the Angular, Next.js and Node standards section for section. Sections 13 and 14 are Python-specific.

Version policy

Projects run a supported Python version — one still receiving security fixes, and never the newest release in its first months. Upgrades are planned work, scheduled at least once a year, not deferred until something forces them.

Enforcement tags

Tag	Meaning
[LINTER]	A tool in the repo catches it — Ruff, mypy.
[CI]	A pipeline step or Git-host setting catches it — Gitleaks, dependency scan, coverage gate, branch protection.
[NEITHER]	A human must judge it in review.
[LINTER + NEITHER]	The linter covers part; review covers the rest.
[CI + NEITHER]	The pipeline covers part; review covers the rest.

Check: notes name the specific tool, or explain why a rule is not automatable. A **backstop** is a narrower rule a tool *can* enforce, used where the real rule cannot be automated.

Roughly half of this document is [NEITHER], and in a backend that half is authorisation, transaction boundaries, idempotency and money. That is the measured reason review is mandatory on every PR.

1. Naming

- [LINTER] Packages, modules and files `snake_case`. Classes `PascalCase`. Functions and variables `snake_case`. Constants `UPPER_SNAKE_CASE`. **Check:** Ruff `N` rules (pep8-naming).
- [LINTER] Files carry a role suffix: `*_route.py`, `*_service.py`, `*_repository.py`, `*_models.py`, `*_schemas.py`, `test_*.py`. *Reason:* a reviewer should know a file's responsibility from the tree alone.
- [NEITHER] Conventional names kept consistent: `db` for the session, `settings` for config, `log` for the logger.
- [NEITHER] **Finite domain values use Enum**, never scattered raw strings. A status compared against a string literal in four places will eventually be compared against a typo.
- [NEITHER] Names state what a thing is, not how it was built. Booleans read as assertions — `is_active`, `has_expired`, `can_withdraw`.
- [NEITHER] Functions returning an optional result start with `find_`; functions that raise when the record is missing start with `get_`. *Reason:* the caller knows the failure mode from the call site.

1.1 Wire format across stacks

[NEITHER] JSON payloads are camelCase on the wire, in both directions. Python stays `snake_case` internally.

Both conventions are correct in their own language. The contract between services is camelCase, and Python is the stack that converts, because it is the one whose internal convention differs from the wire format.

This is configured once, on a base model, not per endpoint:

```
# app/schemas/base.py
from pydantic import BaseModel, ConfigDict
from pydantic.alias_generators import to_camel

class ApiModel(BaseModel):
    """Base for every request and response schema."""
    model_config = ConfigDict(
        alias_generator=to_camel,
        populate_by_name=True,      # accept snake_case too, for internal callers
        from_attributes=True,
    )
```

```
class LoanResponse(ApiModel):
    loan_id: str          # serialises as "loanId"
    principal_minor: str  # serialises as "principalMinor"
    due_date: datetime    # serialises as "dueDate"
```

[LINTER] Every request and response schema inherits `ApiModel`. A schema inheriting `BaseModel` directly emits `snake_case` and breaks the contract. **Check:** Ruff custom rule, or review item 2.

2. Project structure

Feature-first, not layer-first. A feature owns its route, logic, data access, schemas and tests.

```
app/
main.py          # builds the app, registers routers and exception handlers
env.py           # typed Settings – the single source of config
modules/
  loans/
    loans_route.py    # HTTP only – no business logic
    loans_service.py  # business logic
    loans_repository.py # all DB access for this module
    loans_models.py   # ORM table definitions
    loans_schemas.py # request/response models, inheriting ApiModel
    test_loans.py
  common/
    errors.py         # AppError, AppException, the error envelope
    middleware.py     # correlation id, auth, timing
    logging.py        # logger + redaction
    alerting.py       # alert.fire()
    const.py
  database/
alembic/         # engine, get_session dependency, base model
tests/          # the only sanctioned path for schema changes
.env.example    # integration / e2e
```

On the change from layer-first. v1.0 grouped by layer — `app/routes/`, `app/services/`, `app/repository/`. That works, and at small scale it works well. It is changing for new projects for one reason: **a feature is what gets built, reviewed and deleted as a unit**. Under a layer-first tree, adding one feature touches five directories and deleting one means hunting through five. Every other stack in the company landed on feature-first independently, and a developer moving between services should find the same shape.

- [NEITHER] **Feature-first on every new project. No exceptions.** The layout costs nothing on a small project — the same files in differently named folders. **Existing projects are not migrated.** There is no return on that work, and v1.0's layout remains correct for them.
- [NEITHER] **Routes do HTTP. Services decide. Repositories touch the database.** A route containing business logic or raw SQL has crossed a boundary that exists to make the logic testable.
- [NEITHER] **A repository is added when it is earned** — multi-table joins, a second data store, batch or aggregate queries, or a query duplicated across services. Not before. Until then the service uses the session directly.
- [NEITHER] **A service becomes a class once a module has three or more functions sharing the same collaborators** (session, clients). Dependencies are injected, never constructed inside. Do not wrap one function in a one-method class — that is ceremony, not structure.

- [LINTER] No cross-module imports of internals. Modules talk through an exported service. **Check:** Ruff `TID252` plus `flake8-import-restrictions` style configuration.

3. Error handling & validation

- [NEITHER] **Every external input is validated at the boundary** — body, path, query, headers, webhook payloads — with a Pydantic schema. *Reason:* unvalidated input is the single most common source of production incidents.
- [NEITHER] Predicted failures raise `AppException`. Services never build HTTP responses themselves.
- [NEITHER] Exception handlers are wired once in `main.py`. **No raw FastAPI validation detail ever reaches a client.**
- [LINTER] No bare `except:` and no `except Exception` that swallows the error and returns success. **Check:** Ruff `E722`, `BLE001`.
- [NEITHER] Internal error detail, stack traces and driver messages are never returned to the client.
- [NEITHER] Breaking API changes ship under a new version path. Fields are not removed or retyped in place — we do not control all consumers.

3.1 The error contract

Every service in the company returns the same error body. The frontends parse it once.

```
# app/common/errors.py
from enum import StrEnum

class AppError(StrEnum):
    VALIDATION_FAILED = "VALIDATION_FAILED"
    LOAN_NOT_FOUND = "LOAN_NOT_FOUND"
    LOAN_ALREADY_SETTLED = "LOAN_ALREADY_SETTLED"
    FORBIDDEN = "FORBIDDEN"
    INTERNAL_ERROR = "INTERNAL_ERROR"

_STATUS: dict[AppError, int] = {
    AppError.VALIDATION_FAILED: 400,
    AppError.LOAN_NOT_FOUND: 404,
    AppError.LOAN_ALREADY_SETTLED: 409,
    AppError.FORBIDDEN: 403,
    AppError.INTERNAL_ERROR: 500,
}

class AppException(Exception):
    def __init__(
        self,
        code: AppError,
        message: str,
        details: dict[str, list[str]] | None = None,
    ) -> None:
        self.code = code
        self.status = _STATUS[code]
        self.message = message
        self.details = details
        super().__init__(message)
```

Two kinds of invalid deserve two statuses. A malformed request is 400. A well-formed request that conflicts with current state — settling an already-settled loan — is 409. Returning 400 for both means the caller cannot tell a bug from a race.

3.2 Handlers wired once

```
# app/main.py
from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

app = FastAPI()

@app.exception_handler(AppException)
async def handle_app_exception(_: Request, exc: AppException) -> JSONResponse:
    if exc.status >= 500:
```

```

        log.error(exc.message, extra={"code": exc.code})
    else:
        log.warning(exc.message, extra={"code": exc.code})
    return JSONResponse(
        status_code=exc.status,
        content={"code": exc.code, "message": exc.message, "details": exc.details},
    )

@app.exception_handler(RequestValidationError)
async def handle_validation_error(_: Request, exc: RequestValidationError) -> JSONResponse:
    details: dict[str, list[str]] = {}
    for err in exc.errors():
        field = ".".join(str(p) for p in err["loc"][1:]) or "body"
        details.setdefault(field, []).append(err["msg"])
    return JSONResponse(
        status_code=400,
        content={"code": AppError.VALIDATION_FAILED, "message": "Invalid request", "details": details},
    )

@app.exception_handler(Exception)
async def handle_unexpected(request: Request, exc: Exception) -> JSONResponse:
    # Anything unrecognised is ours, and someone needs to look at it.
    log.exception("Unhandled error", extra={"path": request.url.path})
    alert.fire("UNHANDLED_EXCEPTION", {"path": request.url.path})
    return JSONResponse(
        status_code=500,
        content={
            "code": AppError.INTERNAL_ERROR,
            "message": "An unexpected error occurred", # never the real message
            "correlationId": correlation_id.get(),
        },
    )

```

Return the correlation id on a 500. It costs nothing and it is what turns a support ticket into a log lookup. The frontends display it on their error screens.

3.3 Response envelope

- [NEITHER] Success responses carry the resource directly, or `{items, total, page}` for a list. Errors carry `{code, message, details?}`.
- [NEITHER] The HTTP status carries the category; `code` carries the specific case. Clients cannot branch reliably on prose messages.
- [NEITHER] **Never HTTP 200 with `{"success": false}`**. Status codes exist for this.
- [NEITHER] A new error shape outside `AppException` blocks the merge.

4. Logging, observability & alerting

4.1 Log levels

The same four meanings across every stack in the company.

Level	Meaning
error	A human needs to look at this.
warning	Degraded but handled.
info	A state change worth auditing.
debug	Off in production.

One `info` line per database call means nobody reads `info` at all.

4.2 Logs go to stdout, not to a file

[NEITHER] **Structured JSON logs are written to stdout. Services do not write log files.**

v1.0 specified daily files at `logs/YYYY-MM-DD.log`. That changes, for three reasons:

1. **Nobody reads log files daily**, so an error that only reaches a file is an error nobody knows about until a client reports it.
2. **In a container the file disappears on restart** — precisely when you most want it.
3. A `logs/` directory inside the repository is build output in source control.

The platform collects stdout. `logs/` is removed from the tree and added to `.gitignore`.

```
# app/common/logging.py
import logging, json, sys
from app.common.context import correlation_id

DENY = {
    "authorization", "password", "passwd", "secret", "token", "access_token",
    "refresh_token", "api_key", "apikey", "private_key", "mnemonic", "seed",
    "otp", "pin", "cvv", "card_number", "aadhaar", "pan", "ssn", "email", "phone",
}
REDACTED = "[REDACTED]"
MAX_DEPTH = 6

def redact(value: object, depth: int = 0) -> object:
    """Enforced in code. A denylist in a document is a hope."""
    if depth > MAX_DEPTH:
        return "[TRUNCATED]"
    if isinstance(value, dict):
        return {
            k: (REDACTED if k.lower() in DENY else redact(v, depth + 1))
            for k, v in value.items()
        }
    if isinstance(value, (list, tuple)):
        return [redact(v, depth + 1) for v in value]
    return value

class JsonFormatter(logging.Formatter):
    def format(self, record: logging.LogRecord) -> str:
        payload = {
            "level": record.levelname.lower(),
            "message": record.getMessage(),
            "logger": record.name,
            "correlationId": correlation_id.get(),
            "service": settings.SERVICE_NAME,
            "environment": settings.ENVIRONMENT,
        }
        extra = getattr(record, "extra", None) or {
            k: v for k, v in record.__dict__.items()
            if k not in logging.LogRecord(" ", 0, " ", 0, " ", (), None).__dict__
        }
        payload |= redact(extra) # type: ignore[operator]
        if record.exc_info:
            payload["exception"] = self.formatException(record.exc_info)
        return json.dumps(payload, default=str)

handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(JsonFormatter())
log = logging.getLogger(settings.SERVICE_NAME)
log.addHandler(handler)
log.setLevel(settings.LOG_LEVEL)
```

[NEITHER] **Never logged, in any environment:** passwords, tokens, API keys, `.env` values, private keys, seed phrases, OTPs, `Authorization` headers, card or bank data, full PII, KYC payloads.

If something sensitive does reach a log, treat it as a leaked credential and rotate it. The log has already been shipped and retained. Removing the log line changes nothing.

[LINTER] No `print()` for diagnostics. **Check:** Ruff `T20`.

4.3 Correlation IDs

[NEITHER] Every request carries a correlation id through every log line, downstream call and response header.

Reason: without it, tracing one user's request across services is guesswork. Our frontends generate the id and send `X-Correlation-Id`; we honour it when present and mint one when absent.

```
# app/common/context.py
from contextvars import ContextVar
correlation_id: ContextVar[str | None] = ContextVar("correlation_id", default=None)
```

```
# app/common/middleware.py
from uuid import uuid4

@app.middleware("http")
async def correlation_middleware(request: Request, call_next):
    cid = (request.headers.get("X-Correlation-Id") or str(uuid4()))[:64]
    token = correlation_id.set(cid)
    try:
        response = await call_next(request)
        response.headers["X-Correlation-Id"] = cid
        return response
    finally:
        correlation_id.reset(token)
```

[NEITHER] Outbound calls to our own services forward the header. A chain that drops it at the second hop is no more useful than having none.

4.4 Alerting — a log line is not an alert

[NEITHER] Every service has an error alerting system. Nobody watches a log stream, so a failure that only reaches the logs is a failure nobody knows about until a client reports it.

Minimum events that must raise an alert:

Event	Why
Unhandled exception	The service is degraded right now
Failed background or scheduled job	Silent, and compounds daily
Failed third-party API or webhook processing	Our data is now out of sync with theirs
Database connection loss or pool exhaustion	Everything downstream is about to fail
Authentication or payment failures above a threshold	Either an outage or an attack

Alerts carry enough context to act on: environment, service, correlation id, error code and a short message. An alert that only says "Error" costs more time than it saves.

```
# app/common/alerting.py
import time
import sentry_sdk

_WINDOW_SECONDS = 300
_seen: dict[str, float] = {}

def _should_send(key: str) -> bool:
    """Rate-limited so a repeating failure does not flood the channel and get muted."""
    now = time.time()
    last = _seen.get(key)
    if last and now - last < _WINDOW_SECONDS:
        return False
    _seen[key] = now
    return True

class Alert:
    def fire(self, event: str, context: dict[str, object] | None = None) -> None:
        payload = {
            "event": event,
            "environment": settings.ENVIRONMENT, # channels separated by environment
            "service": settings.SERVICE_NAME,
            "correlationId": correlation_id.get(),
            **(context or {}),
        }
```

```

log.error(f"ALERT {event}", extra=payload)
sentry_sdk.capture_message(f"ALERT {event}", level="error")

if not _should_send(f"{event}:{payload.get('code', '')}"):
    return
notify_channel(payload)          # Teams webhook for this environment

alert = Alert()

```

- [NEITHER] **Alert channels are separated by environment.** Production noise is never mixed with dev or staging, or production alerts get muted along with the noise.
- [NEITHER] Sentry is initialised at startup with `send_default_pii=False` and a `before_send` hook that runs `redact()` over the event.
- [NEITHER] **Production minimum:** a health endpoint, error alerting as described, and uptime monitoring.

OpenTelemetry remains the target for unified tracing, adopted when touching a file rather than as a rewrite. It does not replace alerting — a trace tells you what happened after someone already knows to look.

5. Testing expectations

The developer proves the code works before it reaches QA. QA validates the feature; the developer proves it runs.

- [NEITHER] **The developer tests their own work before raising the PR.** A PR is not a request for someone else to find out whether it runs.
- [NEITHER] **Unit tests are required for:** money, amount, interest and fee calculations; date and schedule logic; state-transition rules; shared utilities. *Reason:* these produce a wrong answer silently instead of failing, so no manual tester will catch them.
- [NEITHER] Every endpoint is verified for its success path, its auth-failure path, and its validation-failure path before handover.
- [NEITHER] Failure cases are tested against realistic data: invalid input, missing fields, expired session, third-party down, insufficient balance, duplicate request.
- [NEITHER] **Every project has a `tests/` directory and a runnable suite.** v1.0 said “pytest wherever `tests/` exists”, which permits a project with no tests to satisfy the standard. A new project without tests is not done.
- [NEITHER] Every fixed bug ships with a test that fails without the fix.
- [CI] **Repository tests run against PostgreSQL via Testcontainers, not SQLite.** SQLite accepts queries Postgres rejects, so a green SQLite test proves nothing about production.
- [CI] Tests never run against live third-party APIs or production data.
- [CI] New and changed lines meet the project's coverage threshold; overall coverage must not drop in a PR. **Check:** `pytest-cov` plus a diff-coverage step. *A percentage target on legacy code is meaningless; a ratchet on new code is not.*
- [CI] Existing tests pass before merge. A developer does not skip or delete a failing test to get a PR through.

6. PR & merge rules

- [CI] **One approval minimum, from someone other than the author, on every PR in every project.** Two approvals when the change touches authentication, payments, migrations, or `common/`. **Check:** branch protection plus CODEOWNERS.
- [CI] Blocks merge: failing CI, lint or type errors, self-approval, protected-branch bypass, unresolved blocking comments, dropped coverage, a committed secret, a schema change without a migration, a session constructed inside a service, or an undocumented change to the error response shape.
- [NEITHER] One concern per PR. **Target 400 changed lines**, excluding lock files and generated code. Larger PRs carry a note saying why and how to review them.
- [LINTER] Branches `feature/<ticket>-<slug>`, `fix/<ticket>-<slug>`. No force-push to a shared branch.
- [CI + NEITHER] **Required PR description fields, all of them:** what changed · why it changed · how it was tested · security/architecture impact · breaking changes · related ticket · deployment requirements.
 - *How it was tested* states what was run and what was checked. **“Tested locally” is not an answer.**
 - *Deployment requirements* lists new or changed environment variables, migrations, scripts to run before or after, and the order. **If nothing is needed, write “None”** — silence is not the same as nothing.
- [CI] **Gitleaks runs on every PR and blocks on findings.** A detected secret blocks the merge and **triggers rotation of that credential**, not just its removal from the diff — once committed, it is in the history and must be treated as leaked.

- [CI] A dependency vulnerability scan runs on every PR — `pip-audit` or Trivy. High and Critical findings block; Medium requires triage. Existing findings are baselined so CI blocks on new problems, not old ones. **The baseline is not regenerated to make a failing pipeline pass.**

On the review requirement. About half the rules here are [NEITHER], and in a backend that half is authorisation, transaction boundaries, idempotency and money. On a project where review is switched off, none of that is checked by anything. Exceptions are granted by the CTO for a named person, not chosen per project.

7. Review checklist

1. [NEITHER] Route stays thin — the service owns the logic?
2. [LINTER] Schemas inherit `ApiModel`, so the wire stays camelCase? (1.1)
3. [NEITHER] **Authorisation checked, not just authentication — can user A act on user B's record?** (8.3)
4. [LINTER] `db` injected via `Depends`, not constructed inline? (13.1)
5. [NEITHER] Failures raise `AppException`, not bare exceptions?
6. [NEITHER] **Money handled per section 12 — no float anywhere near an amount?**
7. [NEITHER] Any model change shipped with an Alembic migration, with a rollback plan?
8. [LINTER] No secrets in the diff; `settings.X` used, never `os.getenv()` in a service?
9. [NEITHER] Logs free of tokens and PII?
10. [NEITHER] Queries indexed, bounded, paginated and free of N+1?
11. [NEITHER] **Blocking call inside an async def?** (14.1)
12. [NEITHER] Multi-step writes inside a transaction? (13.2)
13. [NEITHER] Response envelope unchanged, unless intended?
14. [NEITHER] Tests present for the logic changed, including failure paths?
15. [NEITHER] Does this change need a new alert, and is it wired? (4.4)
16. [NEITHER] `.env.example`, README, migrations and deployment steps updated?
17. [NEITHER] Could the newest developer on the team read this without asking the author?

8. Security & secrets

8.1 Secrets

- [LINTER] **Secrets flow only through `.env` → `app/env.py`.** Never `os.getenv()` or a hardcoded value in application code. **Check:** Ruff `no-restricted` configuration on `os.getenv` and `os.environ` outside `app/env.py`.
- [NEITHER] **Secrets come from the environment with no default fallback.** A default gets shipped, and then gets forgotten.

```
# ✗ this reaches production and nobody notices
JWT_SECRET: str = os.getenv("JWT_SECRET", "dev-secret")

# ✓ the service refuses to start
class Settings(BaseSettings):
    JWT_SECRET: str = Field(min_length=32)      # no default
```

- [CI] Never commit `.env`, keys, credentials, customer data dumps or `venv/`.
- [NEITHER] `.env.example` lists every required key with a dummy value and no real ones.

AWS: production secrets come from Secrets Manager or Parameter Store, read at startup through `Settings`. Rotation is a Secrets Manager concern, not a redeploy.

8.2 Authentication

- [NEITHER] Authentication on every non-public route.

- [NEITHER] Passwords hashed with bcrypt or argon2 — never encrypted, reversible, or logged. *Reason:* reversible storage means one database leak exposes every account.
- [NEITHER] **Everything issued for verification expires:** sessions, access and refresh tokens, OTPs, password-reset and email-verification links, signed URLs. Logout, password change or role change invalidates existing sessions.
- [NEITHER] Refresh tokens rotate on use and the old token is invalidated. A reused refresh token is treated as replay: revoke the session and alert.

8.3 Authorisation — the ownership check

[NEITHER] **Authentication alone does not stop user A reading user B's data.** Every endpoint that touches a record checks ownership or role, in the service, every time.

This is **IDOR**, and it is the most likely real vulnerability in our systems. A valid token for user A, with user B's id in the path, returning user B's record. It needs no tools — a user changes a number in the URL and finds it by accident. Scanners miss it: the request is well-formed, authenticated, and returns 200.

The fix is in how you query, not in a check you remember to add:

```
# ✗ returns anyone's loan to anyone who is signed in
loan = await repo.find_by_id(loan_id)

# ✓ the query cannot return another user's record
loan = await repo.find_by_id_for_owner(loan_id, current_user.id)
if loan is None:
    raise AppException(AppError.LOAN_NOT_FOUND, f"Loan {loan_id} not found")
```

Scope the query to the caller and there is no check to forget.

- [NEITHER] **Return 404, not 403, for a record the caller does not own.** A 403 confirms the record exists, which is itself a disclosure.
- [NEITHER] **Every PR that adds an endpoint names, in its description, how ownership is enforced on it.**

8.4 Request safety

- [NEITHER] Every inbound webhook verifies its signature or HMAC **before the payload is processed**. *Reason:* the URL is not a credential, and an unverified endpoint is an unauthenticated write path into the system.
- [NEITHER] Rate limiting on auth, OTP, and any endpoint that costs money.
- [NEITHER] Security headers, a CORS allowlist — never * with credentials — and request body size limits enabled in production.
- [LINTER] **Parameter binding only.** Never build SQL by string concatenation or f-string. **Check:** Ruff S608.
- [NEITHER] File uploads restricted by size and by **actual content type**, not the filename or the client-declared MIME type. Stored privately, served through expiring signed URLs.
- [LINTER] Ruff's S rules (bandit) run on every PR. **Check:** `ruff check --select S`.

8.5 Never trust the client

[NEITHER] Client-supplied identity, amount, price, role or status is re-derived or re-verified server-side. **Always.** A field the server does not verify is a field the server has delegated to an attacker.

9. Dependencies & configuration

- [CI] **Dependencies are locked.** New projects use uv with pyproject.toml and a committed uv.lock. Existing requirements.txt projects pin with == and commit a compiled lock file. *Reason:* pinned top-level versions still resolve transitive dependencies differently on different days — “works on my machine” is nearly always a lock file problem.
- [NEITHER] Adding a dependency needs a one-line justification in the PR: what it does, why the standard library cannot, its last release date. *Reason:* every package is permanent attack surface and future upgrade work.
- [NEITHER] Unmaintained packages — no release or security fix in roughly two years — are flagged for replacement rather than adopted.
- [NEITHER] Use the stable release, not the newest.
- [LINTER] **All config is read and validated once at startup through app/env.py.** The app fails fast on a missing or malformed variable. *Reason:* better to fail at boot than at 2 a.m. on a code path nobody exercised.

```
# app/env.py
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="forbid")

    ENVIRONMENT: Literal["local", "dev", "staging", "production"]
    SERVICE_NAME: str
    LOG_LEVEL: str = "INFO"
    DATABASE_URL: PostgresDsn
    JWT_SECRET: str = Field(min_length=32)
    SENTRY_DSN: str | None = None
    ALERT_WEBHOOK_URL: str | None = None
    DB_POOL_SIZE: int = 10
    DB_MAX_OVERFLOW: int = 5

settings = Settings()      # raises at import time if the environment is wrong
```

`extra="forbid"` means a typo in a variable name fails at boot rather than silently using a default.

- [LINTER] **Ruff is the sole linter and formatter.** No stacking Black, isort and Flake8 on top.
- [LINTER] **mypy runs in CI.** Every new function signature is typed. Existing untyped modules are baselined, not fixed in bulk.
- [NEITHER] **Alembic is the only path for schema changes.** Never hand-patch production with SQL. An applied migration is never edited.

10. Maintainability & complexity

- [LINTER] Functions stay small enough to read without scrolling and nesting stays shallow (3 levels). *Reason:* deep nesting is where edge cases hide. **Check:** Ruff `C901` complexity, `PLR0912`.
- [NEITHER] **New helper functions only when reused or a genuine domain operation.** Type every new signature.
- [NEITHER] The third repetition of the same logic gets extracted; the second does not. Premature abstraction costs more than duplication.
- [NEITHER] Comments explain **why**, never what. Complex business rules — interest, schedules, fees, state transitions — carry a short rationale.
- [NEITHER] Every README covers setup, environment variables, and how to run locally and run tests against a Docker Postgres.
- [NEITHER] **Performance rules:** no queries inside loops; no unbounded `.all()` — paginate; select only the fields needed; index every field used for filtering, sorting or joining; cap payload sizes; move CPU-heavy work off the request path into a job.

11. What we do NOT do

- [NEITHER] **No float for money, token amounts or interest.** See section 12.
- [NEITHER] Business logic or SQL sitting in a route.
- [LINTER] A session or engine constructed inside a service or route. **Check:** review item 4; backstop — Ruff restriction on `create_engine` outside `app/database/`.
- [NEITHER] New error JSON shapes outside `AppException`.
- [LINTER] `os.getenv()` calls or hardcoded secrets in application code; `print()` used for diagnostics.
- [LINTER] A bare `except Exception` that swallows the error and returns HTTP 200.
- [NEITHER] **A blocking call inside an `async def`.** See 14.1.
- [NEITHER] A second ORM, test runner, or web framework parachuted into a feature PR.
- [NEITHER] A vendor APM SDK in place of OpenTelemetry.
- [NEITHER] **Big-bang rewrites riding on an unrelated ticket.** Converting every module to classes is its own PR, agreed in advance.
- [NEITHER] No multi-step database writes without a transaction. See 13.2.
- [NEITHER] Writing log files from the application. See 4.2.
- [NEITHER] A `utils` module with thirty unrelated exports. That is not a utility.

12. Money and numeric values

[NEITHER] **float** never touches a monetary or token amount. Not in a model, not in a calculation, not in a column, not in a schema.

`0.1 + 0.2` is `0.30000000000000004` in Python as in every other language. On a financial platform that produces defects found by clients, not by us.

Kind	Type	Why
Fiat currency	<code>int</code> minor units, serialised as <code>str</code>	Exact, and matches what Node sends
Token / on-chain	<code>int</code> base units (wei), serialised as <code>str</code>	Exact, and matches the chain
Rates and percentages	integer basis points (<code>interest_bps = 250 = 2.5%</code>)	No decimal to round
Intermediate division	<code>Decimal</code> with an explicit context	Rounding is a decision, not an accident

```
# app/common/money.py
from decimal import Decimal, ROUND_HALF_UP

Minor = int          # 1050 == ₹10.50

def apply_bps(amount: Minor, bps: int) -> Minor:
    """Integer arithmetic throughout. No float, no implicit rounding."""
    if not isinstance(bps, int):
        raise TypeError("bps must be an integer")
    return amount * bps // 10_000

def split_evenly(amount: Minor, parts: int) -> list[Minor]:
    """The remainder goes to the first instalment. Decided once, here."""
    base, remainder = divmod(amount, parts)
    return [base + (1 if i < remainder else 0) for i in range(parts)]

def to_wire(amount: Minor) -> str:
    return str(amount)
```

- [NEITHER] **Amounts cross the wire as strings.** JSON numbers lose precision on large values silently, and the frontends treat money as strings.
- [NEITHER] Database columns for money are `NUMERIC / DECIMAL` or `BIGINT`. Never **FLOAT** or **DOUBLE PRECISION**.
- [NEITHER] Pydantic schemas type money as `str`, never `float`.
- [NEITHER] **Rounding is decided once, written down, and applied in one place** — which direction, at which step, and who absorbs the remainder. Two developers rounding independently is how a ledger stops balancing.
- [NEITHER] The backend is authoritative for every total, fee and interest figure.

Never:

```
total = float(a) + float(b)          # x
fee = amount * 0.025                 # x
paise = round(rupees * 100)          # x rounds a float that is already wrong
```

13. Data access & transactions

13.1 One session per request

[LINTER] A request uses one database session, injected via `Depends(get_session)` and closed when the request ends. A service never constructs a session or engine.

Reason: a session created inside a service and not closed leaks a connection. Under load the pool exhausts, and the failure appears as unrelated timeouts across the whole service — rarely traced back to the one handler that opened it.

```
# app/database/session.py
engine = create_async_engine(
    str(settings.DATABASE_URL),
    pool_size=settings.DB_POOL_SIZE,
    max_overflow=settings.DB_MAX_OVERFLOW,
    pool_pre_ping=True,          # a connection killed by the DB is replaced, not reused
)

AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_session() -> AsyncIterator[AsyncSession]:
    async with AsyncSessionLocal() as session:
        yield session          # closed on the way out, even on an exception
```

```
# ✓ the route owns the session's lifetime; the service receives it
@router.post("/loans/{loan_id}/settle")
async def settle_loan(
    loan_id: str,
    db: Annotated[AsyncSession, Depends(get_session)],
    current_user: Annotated[User, Depends(get_current_user)],
) -> LoanResponse:
    loan = await LoanService(db).settle(loan_id, current_user)
    return LoanResponse.model_validate(loan)
```

- **[NEITHER] Pool size is configured deliberately and documented.** The default is rarely right for the instance size, and pool exhaustion is an alerting event (4.4).
- **[NEITHER] Long-running work** — a report, a batch job — does not hold a request-scoped session. It opens its own, outside the pool serving request traffic.
- **[NEITHER] One session is passed through the whole request.** A service that opens a second session for the same request can see a different, uncommitted view of the data.

13.2 Transactions

[NEITHER] Multiple writes that must succeed or fail together go in one transaction.

If a failure partway through would leave related records inconsistent — money moved but status not updated, a record created but a linked one missing — the writes go in a transaction. *A partial write is a data-integrity bug that surfaces days later as a support ticket nobody can explain.*

```
async with db.begin():
    loan.status = LoanStatus.SETTLED
    db.add(LedgerEntry(loan_id=loan.id, amount_minor=payoff))
    # commits on exit, rolls back on any exception
```

- **[NEITHER] Never hold a transaction open across an external API call.** The connection is held for the whole remote timeout and the pool drains. Do the remote call first, or record intent and reconcile.
- **[NEITHER] A migration ships with a rollback plan and is safe to run on a populated table.**

13.3 Idempotency

[NEITHER] Any operation that can be retried carries an idempotency key, and a repeat returns the original result rather than performing the work again.

Clients retry. Load balancers retry. Payment providers and chains redeliver.

- **[NEITHER] The key and the result are written in the same transaction as the work.** Written afterwards, a crash in between leaves the work done and the key missing.
- **[NEITHER] Webhook handlers key on the provider's event id and tolerate duplicates.**

14. Async correctness & background work

14.1 Nothing blocking inside an `async def`

[NEITHER] A blocking call inside an `async def` stops the entire event loop, not just that request. Every concurrent request on that worker waits.

This is the most common and least visible performance defect in a FastAPI service. It does not error. It does not appear in a log. It shows up as latency that nobody can attribute.

```
# ✗ every one of these blocks the loop for all requests on this worker
@router.get("/report")
async def report():
    data = requests.get(url).json()      # blocking HTTP
    rows = pd.read_csv(path)             # blocking file IO
    time.sleep(2)                        # blocking sleep
    hashed = bcrypt.hashpw(pw, salt)     # CPU-bound, ~100ms
```

```
# ✓ async libraries where they exist
@router.get("/report")
async def report():
    async with httpx.AsyncClient() as client:
        data = (await client.get(url, timeout=10)).json()
    await asyncio.sleep(2)

# ✓ a threadpool where they do not
from fastapi.concurrency import run_in_threadpool
hashed = await run_in_threadpool(bcrypt.hashpw, pw, salt)
```

- **[NEITHER]** A route defined with `def` rather than `async def` is run in a threadpool by FastAPI automatically. If a handler is unavoidably blocking end to end, declaring it `def` is correct and safer than an `async def` full of blocking calls.
- **[NEITHER]** Every outbound HTTP call sets a **timeout**. A call without one can hang until the worker is recycled.
- **[NEITHER]** Use the async database driver (`asyncpg`) with `async def` routes. A synchronous driver inside an async route blocks the loop on every query.

14.2 Background and scheduled work

- **[NEITHER]** A failed background or scheduled job raises an alert (4.4). A job that fails silently compounds daily and is found by a client.
- **[NEITHER]** Scheduled jobs are idempotent. A job that runs twice — because the scheduler retried, or two instances started — must not double-process.
- **[NEITHER]** `BackgroundTasks` is for short, best-effort work only. Anything that must complete goes to a real queue, because a `BackgroundTask` dies with the process.
- **[NEITHER]** Long jobs log progress and have a timeout. A job with no timeout and no output is indistinguishable from a hung one.

Appendix — Ruff starting point

Baseline existing violations rather than fixing them all at once.

```
# pyproject.toml
[tool.ruff]
target-version = "py312"
line-length = 100

[tool.ruff.lint]
select = [
    "E", "W",      # pycodestyle
    "F",           # pyflakes
    "I",           # isort
    "N",           # pep8-naming
    "UP",          # pyupgrade
    "B",           # bugbear
    "C4",          # comprehensions
    "C90",         # mccabe complexity
    "S",           # bandit (security)
    "T20",         # no print()
    "BLE",         # no blind except
    "ASYNC",       # async correctness – section 14
    "PL",          # pylint subset
    "RUF",
]
ignore = ["E501"]      # line length is handled by the formatter
```

```

[tool.ruff.lint.per-file-ignores]
"tests/**"      = ["S101"] # assert is the point of a test
"app/env.py"    = ["S105"] # settings field names look like secrets to bandit
"scripts/**"    = ["T20"]  # print is fine in a script

[tool.ruff.lint.mccabe]
max-complexity = 10

[tool.ruff.lint.flake8-tidy-imports]
ban-relative-imports = "all"

[tool.mypy]
python_version = "3.12"
strict = true
warn_unused_ignores = true
# Existing untyped modules are listed here and removed as they are typed.
[[tool.mypy.overrides]]
module = ["app.legacy.*"]
ignore_errors = true

```

ASYNC is the rule set that catches part of section 14.1 — blocking calls in async functions that Ruff can recognise. It does not catch all of them, which is why review item 11 exists.